IEOR 221 - HW 8 - Q2 - Jie Wen

Python script:

```python
import math

def crr_american_call(S0, K, T, r, q, sigma, steps):
    N = steps
    dt = T / N
    disc = math.exp(-r * dt)
    u = math.exp(sigma * math.sqrt(dt))
    d = 1.0 / u
    pu = (math.exp((r - q) * dt) - d) / (u - d)

    # stock prices & option values at maturity
    ST = [S0 * (u ** j) * (d ** (N - j)) for j in range(N + 1)]
    V = [max(s - K, 0.0) for s in ST]

    # keep layers 0..2 for Greeks
    layer_vals = {N: V}
    layer_stks = {'S' + str(N): ST}

    # backward induction with American exercise
    for i in range(N - 1, -1, -1):
        ST_i = [S0 * (u ** j) * (d ** (i - j)) for j in range(i + 1)]
        Vnext = layer_vals[i + 1]
        V_i = []
        for j in range(i + 1):
            cont = disc * (pu * Vnext[j + 1] + (1 - pu) * Vnext[j])
            exer = max(ST_i[j] - K, 0.0)
            V_i.append(max(cont, exer))
        layer_vals[i] = V_i
        layer_stks['S' + str(i)] = ST_i

    price = layer_vals[0][0]

    # Tree-based Greeks from early layers
    V1 = layer_vals[1]; S1 = layer_stks['S1']
    delta = (V1[1] - V1[0]) / (S1[1] - S1[0])

    V2 = layer_vals[2]; S2 = layer_stks['S2']
    Delta_up = (V2[2] - V2[1]) / (S2[2] - S2[1])
    Delta_dn = (V2[1] - V2[0]) / (S2[1] - S2[0])
    gamma = (Delta_up - Delta_dn) / (S1[1] - S1[0])

    # Central time-difference theta (per year)
    theta = (layer_vals[2][1] - layer_vals[0][0]) / (2 * dt)

    return price, delta, gamma, theta
```

```python
def price_only(S0, K, T, r, q, sigma, steps):
    return crr_american_call(S0, K, T, r, q, sigma, steps)[0]

def vega_fd(S0, K, T, r, q, sigma, steps, h=1e-4):
    # dPrice/dSigma
    up = price_only(S0, K, T, r, q, sigma + h, steps)
    dn = price_only(S0, K, T, r, q, sigma - h, steps)
    return (up - dn) / (2 * h)

def rho_fd(S0, K, T, r, q, sigma, steps, h=1e-4):
    # dPrice/d(r)
    up = price_only(S0, K, T, r + h, q, sigma, steps)
    dn = price_only(S0, K, T, r - h, q, sigma, steps)
    return (up - dn) / (2 * h)

if __name__ == "__main__":
    S0 = 100.00
    K = 100.00
    T = 1.00
    r = 0.055
    q = 0.03
    sigma = 0.35
    steps = 100

    price, delta, gamma, theta = crr_american_call(S0, K, T, r, q, sigma, steps)
    vega = vega_fd(S0, K, T, r, q, sigma, steps, h=1e-4)
    rho  = rho_fd (S0, K, T, r, q, sigma, steps, h=1e-4)

    print(f"American Call (CRR, N={steps})")
    print(f"Price : {price:.4f}")
    print(f"Delta : {delta:.4f}")
    print(f"Gamma : {gamma:.4f}")
    print(f"Theta : {theta:.4f}  # per year")
    print(f"Vega  : {vega:.4f}  # per 1.00 vol unit")
    print(f"Rho   : {rho:.4f}   # per 1.00 rate unit")
```

Outputs:

```
American Call (CRR, N=100)
Price : 14.5185
Delta : 0.5801
Gamma : 0.0109
Theta : -7.3013  # per year
Vega  : 37.5505  # per 1.00 vol unit
Rho   : 42.7236   # per 1.00 rate unit
```